# מבוא למדעי המחשב (67102) - סמסטר קיץ, מבחן מועד א'

## מרצה הקורס: גיא אמיר

## מבחן ממוחשב (1/10/2021)

### קראו בזהירות את ההנחיות!

משך הבחינה:
**3 שעות**, לא כולל תוספת זמן עבור הסטודנטים הזכאים לכך.

מבנה הבחינה:
- **חלק א'** - **10 שאלות קצרות** (4 נק' כל אחת). **יש לענות על כל השאלות הקצרות.**
- **חלק ב'** - **6 שאלות תכנות** (15 נק' כל אחת). **יש לענות רק על 4 מתוך 6 השאלות**. בסוף המבחן תתבקשו לסמן אילו שאלות בחרתם.

הנחיות נוספות:
- בשאלות התכנות עליכם לכתוב קוד **יעיל** (מבחינת זמן ריצה וזיכרון) ונקי ככל האפשר. תוכניות לא יעילות או שאינן כתובות היטב לא יקבלו את מלוא הנקודות גם אם הן עובדות נכון.
- בכל שאלות התכנות מותר לכתוב פונקציות עזר נוספות ללא הגבלה. **אך אין לשנות את חתימת הפונקציות (!)**
- אין צורך לתעד את הקוד. אין צורך ב-docstring. הערות לקוד במקומות בהם כתבתם משהו מורכב עשויות לסייע בבדיקת התשובה שלכם אך אינן הכרחיות.
- במידה ותענו על יותר מ-4 שאלות בחלק ב', ייבדקו 4 השאלות הראשונות.
- **הבחינה היא עם חומר סגור. אין לעשות שימוש במחשבון או בדפי נוסחאות, או בכל חומר עזר אחר.**
- מותר לעשות שימוש בדפי טיוטא חלקים. אין צורך להגיש אותם בסוף הבחינה.
- בשאלות השלמת קוד בחלק א', **אין קשר בין אורך המקטע שמסמן קוד חסר, לבין כמות הקוד שצריך למלא**.
- אנא המנעו משאלות לסגל הקורס, אשר אינן קשורות ישירות להבנת השאלה. צוות הקורס לא יוכל לספק רמזים או לענות על שאלות אחרות במהלך המבחן.

### בהצלחה!

# חלק א' - שאלות סגורות

(40% - יש לענות על **כל 10 השאלות**, לכל שאלה 4 נק')

1. להלן פונקצית גנרטור רקורסיבי, אשר מקבל קלט **k** שלם אי-שלילי כלשהו

```
def f(k):
    if k>0:
        yield from f(k-1)
        yield k
        yield from f(k-1)

res = len(list(f(n)))
```

כתבו ביטוי שקול ל-**res**, כפונקציה של **n**, וללא שימוש בפונקציה **f**:

**res =** _____

**solution: 2**(n) - 1**

2. מה סיבוכיות זמן הריצה של הפונקציה הבאה, כתלות בקלט **lst**, שהינה רשימה (באורך **n**) של טיפוסים מסוג **int**.

```
def f2(lst):
     for elmnt in lst:
          if (elmnt % 1) == 0:
               print (“hi”)

          elif (elmnt % 2) == 0:
               for i in range(len(lst)):
                    print (“hi”)

          else:
               return
```

**solution: O(n)**

3. מה סיבוכיות זמן הריצה של הפונקציה הבאה, כפונקציה של **n** אי-שלילי כלשהו:

```
import math

def f3(n):
     s = (n/2) * (n/3) * (n/4) * (n/5)
     for i in range (int(s * math.log(n^2))):
          if i*i < s + 3:
               print (n)
          else:
               break

     for j in range(2 * n):
          for t in range(j):
               print(n)
```

**solution: O(n**2)**

4. לפניכם קוד חלקי של פונקציה **subsets_with_sum** אשר מקבלת מספר כלשהו **n** ורשימת מספרים שונים זה מזה **lst** ומחזירה את מספר תת-הקבוצות השונות, שניתן להרכיב מאברי **lst** ואשר נסכמים ל-**n**.

**למשל:** עבור הרשימה **lst=[1,2,3,5,7]**

- עבור המספר **n=8** אז נקבל

```
subsets_with_sum(8, lst) -> 3
```

כי יש 3 תת-קבוצות אשר איבריהן מ-lst והן נסכמות ל-8: **{1 , 7} , {3 , 5} , {1 , 2 , 5}**

- ועבור המספר **n=0** אז נקבל

```
subsets_with_sum(0, lst) -> 1
```

כי יש רק תת-הקבוצה הריקה (קרי, [])נסכמת ל-0: **{ }**

- ועבור המספר **n=-7** אז נקבל

```
subsets_with_sum(-7, lst) -> 0
```

כי אין אף תת-קבוצה של מספרים שתסכם למספר שלילי עבור הליסט הנ"ל

השלימו את הפונקציה הרקורסיבית הבאה, כדי שהפונקציה תהיה נכונה.

```
def subsets_with_sum(n,lst):
```

```
    if not lst: return 1 if ______ else 0
    return ________________________
```

**solution:**

```
def subsets_with_sum(n,lst):
    if not lst: return 1 if n==0 else 0
    return subsets_with_sum(n,lst[1:])+\
                                subsets_with_sum(n-lst[0],lst[1:])
```

5. לפניכם קוד חלקי של פונקציה **count_larger_than_n** אשר מקבלת מספר אי-שלילי **n** ורשימת מספרים **lst**. הפונקציה צריכה להחזיר את מספר האיברים מתוך **lst** אשר גדולים ממש מ-**n**. השלימו את הקוד החסר.

```
from functools import reduce

def count_larger_than_n (n, lst):
     filter_result = filter(lambda _____ : ________, lst)
     return reduce(lambda _____ : ________, filter_result, ______)
```

**Soulution:**

```
from functools import reduce


def count_larger_than_n (n, lst):

     filter_result = filter(lambda x: x >n, lst)
     return reduce(lambda count, item: count+1, filter_result, 0)
```

6. לפניכם קוד חלקי של המחלקה **ReverseEnumerateIterator**. המחלקה מממשת איטרטור המקבל מחרוזת **string** כלשהי ומחזירה בכל קריאה ל-**next** את הטאפל

**(character last character, i-th last index of i-th)**

כלומר טאפל עם האינדקס של האות ה-i מסוף המחרוזת, ואת האות ה-i מסוף המחרוזת.

אסור להשתמש בפונקציות **reversed** או **enumerate**.

**למשל:**

```
it = ReverseEnumerateIterator("abc")
it.next() -> (2, c)
it.next() -> (1, b)
it.next() -> (0, a)
```

השלימו את הקוד החסר.

```
class ReverseEnumerateIterator:
   def __init__(self, string):
       self.x = ______

   def __iter__(self):
       return self

   def __next__(self):
       if ______:
           raise StopIteration
       return ______
```

**solution:**

```
class ReverseEnumerateIterator:
   def __init__(self, string):
       self.x = list(string)

   def __iter__(self):
       return self

   def __next__(self):
       if not self.x:
           raise StopIteration
       return len(self.string)-1,self.string.pop()
```

7. לפניכם קוד חלקי של הפונקציה **linked_range(start,end)** אשר מקבל זוג מספרים שלמים **start** ו-**end** ובונה רשימה מקושרת בה מופיעים ברצף ולפי הסדר המספרים בטווח **range(start, end)**. אם **start**=>**end**, הפונקציה תחזיר **None**.

ניתן (וצריך) להשתמש במחלקה **Node**, ואין להשתמש בלולאות.

```
def linked_range(start, end):
    if __________:
        return ________________
```

```
class Node:
     def __init__(self, data, next=None):
     self.data = data
     self.next = next
```

**למשל:**

```
>> linked_range(2, 5) -> return Node(2, Node(3, Node(4)))
>> linked_range(2, 2) -> return None
```

**solution:**

```
def linked_range(start, end):
    if start < end:
        return Node(start, linked_range(start + 1, end))
```

8. לסדרת הספרות של מספר שלם נקרא סדרת "פסגות מקסימאלית", אם כל ספרה במספר, קטנה ממש או גדולה ממש מהספרות השכנות שלה. מספר בעל ספרה אחת, מהווה סדרת "פסגות מקסימאלית".

השלימו את הביטוים החסרים במקומות המסומנים בפונקציה **max_peaks(n)**, אשר מקבלת מספר שלם חיובי **n** ומחזירה **True** אם הספרות של n מהוות סדרת פסגות מקסימאלית, ו **False** אחרת. בתשובות, אין להשתמש ב **int, str**, וגם לא באינדקסים או **slicing**.

```
def max_peaks(n):
    def rec_peaks(n,val):
        if _______: # (1)
```

```
            return True
        elif val:
            return ______ and rec_peaks(n//10, not val) # (2)
        else:
            return ________ and rec_peaks(n//10, not val) # (3)
    return rec_peaks(n,True) or rec_peaks(n, False)
```

דוגמאות:

| | |
|---|---|
| `> max_peaks(1)`<br>`True`<br><br>`> max_peaks(11)`<br>`False` | `> max_peaks(12)`<br>`True`<br><br>`> max_peaks(123)`<br>`False` |
| `>max_peaks(21448)`<br>`False` | `>max_peaks(2817263549)`<br>`True`<br><br>`>max_peaks(21438790)`<br>`True` |

**solution:**

```
def max_peaks(n):
    def rec_peaks(n, val):
        if n<10:
            return True
        elif val:
            return n % 10 < n // 10 % 10 and rec_peaks(n//10, not val)
        else:
            return n % 10 > n // 10 % 10 and rec_peaks(n//10, not val)
    return rec_peaks(n,True) or rec_peaks(n,False)
```

9. לאחר ריצת הקוד הבא, המשתנה res מכיל רשימה (list) שבה מוכלות רשימות אחרות, באופן ישיר (האיברים שבתוך res הם רשימות) או עקיף (באחת מרמות העומק ב-res יש רשימות). כמה רשימות שונות זו מזו יש סה"כ ב-result (כולל res עצמו) באופן ישיר או עקיף?

אין לספור lists שמצביעים עליהם כמה פעמים יותר מפעם אחת (כלומר, בלי ספירת רשומות אשר הן aliasing זו של זו). הזינו את תשובתכם בתיבת הטקסט הבאה. הזינו מספר בלבד, ללא רווחים או הסברים.

```
def recursive_list_maker(k):
   if not k:
       return [k, []]
   result = [k, (k-1)*recursive_list_maker(k-1)]
   return result

res = recursive_list_maker(5)
```

**Solution: 6**

10. מה סיבוכיות זמן הריצה של הפונקציה הבאה, כפונקציה של **n** אי-שלילי כלשהו:

```
def f4(n):
     for i in range(2*n**3):
          t = i * i
          for j in range(t, n**2 +1):
               print(n)
```

**solution: O(n**3)**

# חלק ב' - שאלות תכנות

(60% - יש לענות רק על **4 מתוך 6** השאלות, לכל שאלה 15 נק')

1. ריבוע קסם הינו מטריצה ריבועית (כלומר, טבלה דו-מימדית עם מספר שורות אשר שווה למספר העמודות) בה סכום כל שורה, שווה לסכום כל עמודה, ששווה לסכום כל אלכסון. בשאלה זאת ניתן להניח שהמטריצה מכילה את כל הערכים השלמים בטווח שבין **1** ל-**n**2** (כאשר **n** הינו מספר השורות / עמודות בריבוע), וכן שכל ערך בטווח זה מופיע במטריצה פעם אחת בדיוק.

כתבו פונקציה **is_magic_square** אשר מקבלת מטריצה ריבועית **m**, ומחזירה **True** אם המטריצה הריבועית הינה "ריבוע קסם", ו-**False** אחרת. נזכיר שכל מטריצה מיוצגת על ידי רשימה של רשימות. ניתן להניח שהקלט תקין (דהינו, מתקבלת מטריצה ריבועית של מספרים).

חשוב - ניתן להניח שהערכים של המטריצה **m** הינם ערכים שלמים רצופים.

למשל:

```
square_1 = [[1,2,3],[4,5,6],[7,8,9]]
```

מייצגת את הריבוע, הבא אשר לא ריבוע קסם:

| | | |
|---|---|---|
| 1 | 2 | 3 |
| 4 | 5 | 6 |
| 7 | 8 | 9 |

ולכן:

```
is_magic_square(square_1) -> False
```

וכן עבור הריבוע הבא:

```
square_2 = [[4,9,2],[3,5,7],[8,1,6]]
```

מייצגת את הריבוע, הבא אשר כן ריבוע קסם:

| | | |
|---|---|---|
| 4 | 9 | 2 |
| 3 | 5 | 7 |
| 8 | 1 | 6 |

מכיוון שסכום כל שורה שווה לסכום כל עמודה, אשר שווה לסכום כל אלסון, אשר שווים ל-15:

ולכן:

```
is_magic_square(square_2) -> True
```

**הצעה לפתרון:**

```
def is_magic(m):
   if m == [[]]:
       return True
   n = len(m)
   s = (n ** 2 + 1) * n // 2
   all_num = set(range(1, n ** 2 + 1)) == set(i for r in m for i in r)
   rows_eq = all(sum(r) == s for r in (*m, *zip(*m)))
```

```
    diagonals_eq = sum(m[i][i] for i in range(n)) == sum(m[i][-i - 1]  \
                                                    for i in range(n)) == s
    return (all_num and rows_eq and diagonals_eq)
```

2. לפניכם המחלקה **TreeNode** שמייצגת קודקוד בעץ חיפוש בינארי:

```
class TreeNode:

    def __init__(self, value, left=None, right=None):
        self.value = value
        self.left = left
        self.right = right
```

נזכיר שעץ חיפוש בינארי מקיים שעבור כל קודקוד בעץ, כל הערכים בתת העץ השמאלי קטנים יותר מהקודקוד, וכל הערכים בתת העץ הימני גדולים יותר מהקודקוד.

כתבו פונקציה **closest_value(root, target)** אשר מקבלת:

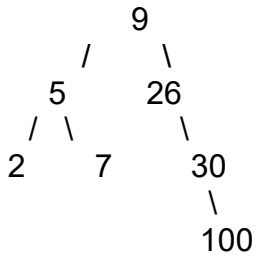
- **root** - שורש של עץ חיפוש בינארי (אובייקט מסוג **TreeNode**) עם ערכים ייחודיים
- **target** - מספר כלשהי, אשר לאו דווקא מופיע בעץ

הפונקציה תחזיר את הערך בעץ, אשר הכי קרוב ל-**target** (אם יש כמה אפשרויות, ניתן להחזיר כל אחת מהן)

למשל:

```
    root = TreeNode(9)
    root.left = TreeNode(5)
    root.right = TreeNode(26)
    root.left.left = TreeNode(2)
    root.left.right = TreeNode(7)
    root.right.right = TreeNode(30)
    root.right.right.right = TreeNode(100)
```

מגדיר את העץ:




```
closest_value(root, 1) -> 2 # in distance 1 from target
closest_value(root, 2) -> 2 # in distance 0 from target
closest_value(root, 8) -> 7 or 9 # in distance 1 from target
closest_value(root, 29) -> 30 # in distance 1 from target
closest_value(root, 80) -> 100 # in distance 20 from target
closest_value(root, 105) -> 100 # in distance 5 from target
```

**הצעה לפתרון:**

```
def closest_value(root, target):
    a = root.val
    son = root.left if target < a else root.right
    if not son:
        return a
    b = closest_value(son, target)
    return min((a,b), key=lambda x: abs(target-x))
```

3. כתבו פונקציה **is_rotation** אשר מקבלת שתי מחרוזות (סטרינגים) כלשהן **s1** ו-**s2** וצריכה להחזיר **True** אם **s1** ו-**s2** הן רוטציה אחד של השני (ו-**False** אחרת).

למשל -

```
is_rotation(“”, “”) -> True
is_rotation(“a”, “”) -> False

is_rotation(“intro”, “troin”) -> True
is_rotation(“intro”, “rotin”) -> False

is_rotation(“hello”, “llohe”) -> True
is_rotation(“hello”, “ollhe”) -> False
```

שימו לב -

- מותר להשתמש רק פעם אחת לכל היותר בפונקציה **in** של פייתון
- אסור להשתמש ברקורסיה או בלולאות משום סוג
- חובה שהפתרון יהיה עם סיבוכיות זמן ריצה של **O(n)** וכן סיבוכיות זיכרון של **O(n)**

**הצעה לפתרון:**

```
def is_rotation(s1, s2):
    length1, length2 = len(s1), len(s2)
    if s1=="">=s2:
        return True
    if length1 == length2 and length1 > 0:
        double_s1 = s1 + s1
        return s2 in double_s1
    return False
```

4. כתבו גנרטור \ איטרטור אינסופי בשם **lexico_iter**, אשר מקבל כקלט רשימה **lst** ממוינת ,ולא ריקה, של תווים בודדים (לפי סדר הא"ב), ויוצרת סדרת סטרינגים לקסיקוגרפית אינסופית, הבנויה לפי איברי הרשימה. כך שהסדרה האינסופית בנויה ממחרוזות (סטרינגים) בסדר הבא:

- תחילה, תופיע **המחרוזת הריקה ""**
- לאחר מכן, יופיעו אחת-אחת כל המחרוזות **בגודל תו אחד** (כאשר הסדר הפנימי ביניהן לפי סדר הא"ב)
- אח"כ, יופיעו אחת-אחת כל המחרוזות **בגודל 2 תווים** (כאשר הסדר הפנימי ביניהן הוא לפי סדר הא"ב)
- וכך הלאה

<u>הערה</u> - חובה שהפתרון יהיה רקורסיבי.

<u>למשל</u> - דוגמא לערכים שיוחזרו בקריאות ה-**next** הראשונות, עבור קלטים שונים:

```
lexico_iter (["a"])
> '','a', 'aa', 'aaa', 'aaaa', …

lexico_iter (['a', 'b'])
> '', 'a', 'b','aa' ,'ab','ba', 'bb', 'aaa', 'aab', 'aba', 'abb',
'baa', 'bab', 'bba', 'bbb' …

lexico_iter (['a', 'b', 'c'])
> '','a', 'b', 'c', 'aa', 'ab', 'ac', 'ba', 'bb', 'bc', 'ca', …
```

**הצעה לפתרון:**

```
def lexico_iter(lst):
   n = 0
   yield ""
   while True:
       yield from recursive_helper(n, lst)
       n += 1


def recursive_helper(n, lst):
   for char in lst:
        if not n:
           yield char
        else:
           for value in recursive_helper(n-1, lst):
               yield char + value
```

5. ממשו את הפונקציה **count_strings** אשר מקבלת מספר שלם ואי-שלישי **n** ומחזירה את <u>כמות</u> <u>המחרוזות</u> הבינאריות באורך <u>בדיוק</u> **n**, אשר אינן כוללות שני "1"ים רצופים. נזכיר שמחרוזת בינארית באורך **n** הינה מחרוזת בעלת **n** תווים, אשר מורכבת רק מ"0" ומ"1".

הערה - חובה שהפונקציה תהיה רקורסיבית, או לחלופין תקרא לפונקצית עזר רקורסיבית.

הערה (2) - מחרוזת ריקה (**n=0** תווים) הינה מחרוזת חוקית

למשל:

```
count_strings(0) -> 1
# good strings are: “”

count_strings(1) -> 2
# good strings are: “0”, “1”

count_strings(2) -> 3
# good strings are: “00”, “01”, “10”
# but NOT “11”

count_strings (3) -> 5
# good strings are: “000”, “001”, “010”, “100”, “101”
# but NOT “110”, “011”, “111”
```

**הצעה לפתרון (1):**

```
def count_strings (n):
      if n<2:
            return n+1
      return count_strings(n-1) + count_strings (n-2)
```

**הצעה לפתרון (2):**

```
def count_strings (n):
      return count_helper(n)

def count_helper(n, last_digit=0):
      if n==0:
            return 1

      if n == 1:
            return 1 if (last_digit==1) else 2

      # last digit is 0, we can have both 0 and 1 at current position
      if last_digit==0:
            return count_helper(n-1, 0) + count_helper(n-1, 1)


      # last digit is 1, we can have only 0 at current position
      if last_digit==1:
            return count_helper(n-1, 0)
```

6. כתבו מחלקה **Baker** שכל אובייקט שלה הוא אופה, שמתמחה בהכנת מאפה אחד. למחלקה יהיו שני שדות:
   - המילון **storage** שישמש לאחסון חומרי האפייה המשותפים, וייצג אותם ואת הכמויות העדכניות שלהם - מפתח במילון **storage** הוא שם של רכיב (מחרוזת), והערך: כמות עדכנית של יחידות של הרכיב.
   - הרשימה **shelf** בה שמורים המאפים שכבר הוכנו.

בנוסף יש לכתוב 4 שיטות כפי שיפורט בהמשך.

---

א. הבנאי: **def __init__(self, product, ingredients**)
הבנאי מקבל את שם המאפה שלו - **product**, ורשימת רכיבים הנחוצים לאפייתו - **ingredients**.
ייתכן שיהיו כפילויות ברשימה אם נדרשת יותר מיחידה אחת להכנת המאפה.

---

ב. השיטה : **def purchase(self, parts**)
מקבלת רשימת מצרכים (מצרך יכול להופיע כמה פעמים) ומוסיפה אותם למחסן (**storage**).

---

ג. השיטה: **def bake(self)**
אם הרכיבים הדרושים נמצאים ב-**storage** השיטה תוציא את הרכיבים הדרושים, תוסיף ל-**shelf** את המוצר המוגמר ותחזיר True.
אחרת, יש לזרוק שגיאה (**Exception**) עם הודעה (בהנחה ששם המאפה הוא chocolate cake):
**'Not enough ingredients in storage for chocolate cake'**

---

ד. עליכם לכתוב מתודה שתאפשר הדפסה של אובייקט מטיפוס אופה.
כאשר נקרא ל-**print** על אובייקט מטיפוס **Baker**, אם לדוגמא האופה מכין cookie והמצרכים הנדרשים הם סוכר אחד, שני קמח, וחמאה אחת, תודפס המחרוזת הבאה:
'Bakes cookie - 1 sugar, 2 flour, 1 butter'
בהתאם לשם המאפה ולכמות המצרכים הדרושים להכנתו.
שימו לב לדייק במחרוזת המודפסת.

---

דוגמת הרצה:

```
> dan = Baker('nuts_delight',['nuts','maple','sugar','cider','almonds',
'nuts'])

> dan.purchase(['maple', 'sugar', 'nuts', 'maple', cider'])

> dan.bake()
  Exception: 'Not enough ingredients in storage for nuts_delight'
```

```
> dan.purchase([‘nuts’, ‘almonds’])

> gal = Baker(‘healthy_bounty’, [‘maple’, ‘tahini’, ’coconut’, ‘coconut oil’,
‘dark chocolate’])

> dan.purchase([‘tahini’, ‘tahini’, ‘coconut’, ‘coconut oil’, ‘dark
chocolate’])

> dan.bake()

> Baker.shelf
  [‘nuts_delight’]

> gal.bake()

> Baker.shelf
  [‘nuts_delight’, ‘healthy_bounty’]

> print(dan)
  ‘Bakes nuts_delight - 1 maple 1 sugar 1 cider 1 almonds 2 nuts’

> print(gal)
  ‘Bakes healthy_bounty - 1 maple 1 tahini 1 coconut 1 coconut oil’
```

**הצעה לפתרון:**

```
from collections import Counter

class Baker:
   storage = {}
   shelf = []


   def __init__(self,product,ingredients):
# ingredients may contain repeated items
       self.product=product
       self.ingredients=Counter(ingredients)

   def purchase(self,shopping_list):
       for item in shopping_list:
           if item not in Baker.storage:
               Baker.storage[item]=0
           Baker.storage[item]+=1

   def bake(self):
```

```
    for item,required in self.ingredients.items():
        if Baker.storage.get(item,0) < required:
            raise Exception (f"Not enough ingredients in \
                                        storage for {self.product}")

    for item,required in self.ingredients.items():
        Baker.storage[item]-=required
    Baker.shelf.append(self.product)
    return True

def __str__(self):
    ingredients_to_print = ", ".join([f"{str(count)} {item}" \
                    for item, count in self.ingredients.items()])
    return f"Bakes {self.product} - {ingredients_to_print}"
```